

Deep Learning Practical No.3

Name - Vedant Joshi

Roll No.24

```
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
```

```
fashion_mnist = tf.keras.datasets.fashion_mnist
(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ————— 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ————— 2s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ————— 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ————— 1s 0us/step
```

```
x_train = x_train / 255.0
x_test = x_test / 255.0
```

```
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)
```

```
class_names = [
    "T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
    "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"
]
```

```
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` a
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

```
history = model.fit(x_train, y_train, epochs=10, validation_data=(x_test, y_test))
```

```
Epoch 1/10
1875/1875 ————— 61s 32ms/step - accuracy: 0.8353 - loss: 0.4526 - val_accuracy: 0.8741 - val_loss: 0.3527
Epoch 2/10
1875/1875 ————— 84s 33ms/step - accuracy: 0.8894 - loss: 0.3013 - val_accuracy: 0.8887 - val_loss: 0.3090
Epoch 3/10
1875/1875 ————— 58s 31ms/step - accuracy: 0.9052 - loss: 0.2550 - val_accuracy: 0.8965 - val_loss: 0.2848
Epoch 4/10
1875/1875 ————— 83s 31ms/step - accuracy: 0.9171 - loss: 0.2218 - val_accuracy: 0.9083 - val_loss: 0.2564
Epoch 5/10
1875/1875 ————— 57s 30ms/step - accuracy: 0.9272 - loss: 0.1936 - val_accuracy: 0.9062 - val_loss: 0.2648
Epoch 6/10
1875/1875 ————— 57s 30ms/step - accuracy: 0.9352 - loss: 0.1729 - val_accuracy: 0.9061 - val_loss: 0.2603
Epoch 7/10
1875/1875 ————— 58s 31ms/step - accuracy: 0.9432 - loss: 0.1517 - val_accuracy: 0.9114 - val_loss: 0.2629
Epoch 8/10
1875/1875 ————— 57s 30ms/step - accuracy: 0.9508 - loss: 0.1327 - val_accuracy: 0.9133 - val_loss: 0.2630
Epoch 9/10
1875/1875 ————— 57s 30ms/step - accuracy: 0.9546 - loss: 0.1193 - val_accuracy: 0.9102 - val_loss: 0.2870
Epoch 10/10
1875/1875 ————— 82s 30ms/step - accuracy: 0.9615 - loss: 0.1030 - val_accuracy: 0.9099 - val_loss: 0.2914
```

```
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print(f"\n✅ Test accuracy: {test_acc:.4f}")
```

```
313/313 - 4s - 14ms/step - accuracy: 0.9099 - loss: 0.2914
```

```
✅ Test accuracy: 0.9099
```

```
14 Test history: history={'loss': 0.2914, 'val_loss': 0.2914, 'accuracy': 0.9099, 'val_accuracy': 0.9099}
```

```
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

